

Pylot-BT: Combinatorial Motion Planning for a Four-Head Flying-Probe EV-Battery Tester

Giovanni Cadau

R&D internship, Seica S.p.A., Strambino (TO), Italy — October–December 2019

github.com/gcadau/Pylot-BT · Report revised 2026

Abstract

This report documents the design and implementation of a motion-planning engine for a four-head flying-probe tester for electric-vehicle battery packs, developed as an R&D internship project at Seica S.p.A. The machine class targeted by the prototype—four independent XYZ heads, each carrying a mini-fixture able to probe up to four cells in a single placement—was later commercialized by Seica as the *Pilot BT* flying prober. The planning problem is a set-cover/scheduling hybrid with geometric side constraints: every cell of the pack must be probed, heads must never collide or cross, each fixture placement must geometrically fit a group of cells, and total head travel should be minimized. The engine is written in $\approx 2\,600$ lines of dependency-free C. All data structures (opaque-pointer abstract data types, amortized dynamic arrays, nearest-neighbour structures, combinatorial enumerators) and all algorithms (recursive candidate-group enumeration with set-level deduplication, topology clustering, multi-start depth-first search with backtracking and branch-and-bound pruning) are implemented from scratch. On the committed 444-cell production dataset the baseline planner covers the pack in 92 movements with ≈ 24.9 m of total head travel; the final version explores $\approx 10^4$ start configurations exhaustively while keeping per-step choices greedy-guided. A 2026 retrospective audit with AddressSanitizer, included here, identified and fixed three latent memory-safety defects masked by the original Windows toolchain.

1 Introduction

Electric-vehicle battery packs are assembled from hundreds of cells whose interconnections must be electrically verified at the end of the production line. Flying-probe architectures solve this without per-product fixtures: independently actuated test heads travel over the pack and touch down on groups of cells. The machine class considered here has $H = 4$ independent heads; each head carries a mini-fixture that contacts up to $K = 4$ cells per placement.

The efficiency of such a machine is dominated by its *motion plan*: the sequence of simultaneous head placements that achieves full electrical coverage. Computing a good plan is a combinatorial optimization problem coupling set cover, vehicle-routing-like travel minimization and geometric collision constraints. This project, carried out during a three-month internship embedded in the machine’s R&D phase, built a complete planning prototype from first principles: no external libraries, every structure and algorithm hand-written, developed against a real 444-point pack layout

supplied by the company.

2 Problem formulation

Let $C = \{c_1, \dots, c_n\}$ be the cell set with coordinates $p(c_i) \in \mathbb{R}^2$ ($n = 444$ in the committed dataset), and let $\mathcal{H} = \{h_1, \dots, h_H\}$ be the heads, each with a rectangular footprint, mechanical tolerances and a fixture offset. A *group* $g \subseteq C$, $1 \leq |g| \leq K$, is a candidate single-placement target. A *movement* is an assignment $m : \mathcal{H} \rightarrow \mathcal{G} \cup \{\perp\}$ of one group (or idleness $\perp$) to every head; a *plan* is a sequence $M = (m_1, \dots, m_T)$.

A feasible plan satisfies:

1. **Coverage:** $\bigcup_{t \leq T} \bigcup_h m_t(h) = C$.
2. **Fixture fit:** for every assigned g , the bounding box of $p(g)$ fits within the probe field of the head, tolerances included.
3. **Non-collision / non-crossing:** within each m_t , heads preserve a fixed left-to-right order and the x -projections of their occupied surfaces, centred on the group centroids, are pairwise disjoint.
4. **Topology compatibility:** successive groups assigned to a head belong to one *topology class*—an equivalence class of groups under translation of the relative cell geometry—because the fixture contact pattern is rigid.
5. **Bounded redundancy:** a cell may be re-probed only within a configured allowance; heads may idle when this aids feasibility.

The objective is lexicographic in spirit: minimize the number of movements T and, among short plans, the total centroid travel $\sum_h \sum_t \|\bar{p}(m_{t+1}(h)) - \bar{p}(m_t(h))\|_2$, where $\bar{p}(\cdot)$ denotes a group centroid. Even the coverage core alone (partition of C into fixture-feasible groups of size $\leq K$ across H parallel machines) generalizes set cover, so the problem is NP-hard; the engineering question is how much exactness a real dataset can afford.

3 Software architecture

The codebase is organized as ≈ 20 modules, each exposing an *opaque-pointer abstract data type*: headers declare only an incomplete struct type and its operations, so representation choices are invisible across module boundaries—the C idiom for information hiding. Table 1 summarizes the layer structure.

Table 1: Module layers (all hand-written, libc + libm only).

Layer	Modules and role
Geometry	<code>coordinata</code> , <code>distanza</code> , <code>forma</code> (square / rectangle / rhombus footprints), <code>topologia</code> (shape-equivalence of relative coordinates)
Collections	<code>celle</code> , <code>gruppi</code> , <code>teste</code> : dynamic arrays with geometric growth $\times 2$ (amortized $O(1)$ append); <code>celle</code> additionally precomputes pairwise distances and answers k -nearest-neighbour queries
Entities	<code>cella</code> , <code>gruppo</code> (with <i>phase</i> = number of already-probed member cells), <code>testa</code>
Enumeration	<code>matrix</code> (sentinel-terminated integer vectors with order-insensitive set deduplication), <code>combinatore</code> (mixed-radix cartesian product), <code>partitore</code> (subset lattice of idle-head patterns)
Search state	<code>movimento</code> , <code>soluzione</code> (incumbent plan memorization and serialization)
Sorting	three specialized quicksorts (<code>sortingCelle</code> , <code>sortingGruppi</code> , <code>sortingInt</code>) with injected total orders
I/O	<code>allocazione</code> : resumable generator-style parser whose cursor is threaded through a <code>void**</code> ; the input header line doubles as the head-geometry schema

A structural detail worth noting: topology clustering is realized as a *recursive nested collection*—the `gruppi` container holds child `gruppi` partitions, one per topology class, reusing the same ADT at both levels.

4 Algorithms

4.1 Candidate-group enumeration

Groups are generated by recursive expansion over the k -nearest-neighbour graph of the cells (Algorithm 1). For each cardinality $\kappa = 1, \dots, K$ and each seed cell, a depth-first expansion over the k nearest unvisited neighbours ($k = \text{PIUVICINE}$) emits candidate index vectors; each vector is sorted and inserted only if no permutation of it was seen before, so groups are deduplicated *as sets*. Restricting expansion to the k -NN graph is the key pruning decision: it keeps the candidate pool near-linear in n instead of $\Theta(n^K)$, at the cost of excluding geometrically implausible (spread-out) groups that the fixture-fit filter would reject anyway. Groups failing the fixture-fit test are discarded eagerly; survivors are partitioned into topology classes.

4.2 Plan search

The final planner (v6) layers exhaustive enumeration exactly where its cost is affordable and greedy guidance elsewhere (Algorithm 2).

Three mechanisms bound the search. *Branch-and-bound*: any prefix longer than the incumbent is cut. *Phase guidance*: a group’s phase counts already-probed members, and preferring phase-0 groups at minimal distance keeps redundancy near zero without forbidding it. *Explicit backtracking*: phase

Algorithm 1: Group enumeration (path/pathRic)

```

1 foreach  $\kappa \in \{1, \dots, K\}$  do
2   foreach seed  $u \in C$  do
3      $\text{EXPAND}(u, \langle \rangle, \kappa)$ ;
4 procedure  $\text{EXPAND}(u, \pi, \kappa)$ 
5   append  $u$  to  $\pi$ ;
6   if  $|\pi| = \kappa$  then
7     if  $\text{sorted}(\pi)$  unseen then
8       register group  $\pi$ 
9   else
10    foreach  $v \in k\text{-NN}(u)$ ,  $v \notin \pi$  do
11       $\text{EXPAND}(v, \pi, \kappa)$ 

```

Algorithm 2: Multi-start B&B search (v6)

```

1  $best \leftarrow T_{\max}$ ;  $S^* \leftarrow \emptyset$ ;
2 foreach start assignment  $s$  in Combinatore (cartesian
   product of per-head start candidates) do
3   if  $s$  mechanically compatible then
4     probe  $s$ ;  $\text{DFS}(s, t=2)$ ; reset probe marks;
5 procedure  $\text{DFS}(m_{t-1}, t)$ 
6   if  $t - 1 > best$  then return prune;
7   if all cells probed then
8      $best \leftarrow t - 1$ ; memorize plan; return
9   choose, per head, the nearest same-topology group of
     minimal phase (escalating phase only if required);
10  foreach idle-head pattern in Partitore ( $2^H$ ) do
11    complete/repair the assignment under redundancy
       rules;
12    if mechanically compatible then
13      probe;  $\text{DFS}(m_t, t+1)$ ; return on success;
14    else
15      restore phases (backtrack);

```

mutations are saved and restored around each failed idle-pattern branch, so the shared group pool acts as an undoable search state. On the committed dataset the outer loop enumerates $\approx 10^4$ start configurations. Earlier version 7 made the *inner* choice exhaustive as well; it confirmed that full exhaustiveness does not scale to $n = 444$, which motivated v6’s hybrid design.

5 Iterative development

Development proceeded in two phases visible in the repository history: the data-structure layer first (branches `Struttura-Dati`, `Struttura-Dati_Test`, Nov 11), then eight planner iterations, each frozen on its own branch (Table 2).

6 Results on the committed dataset

The repository preserves the baseline planner’s full output (branch `Algoritmo`, `Dati/Output/`). Re-parsed today, the committed plan covers all 444 cells in **92 movements** with 460 probe operations (16 redundant re-probes), a mean of

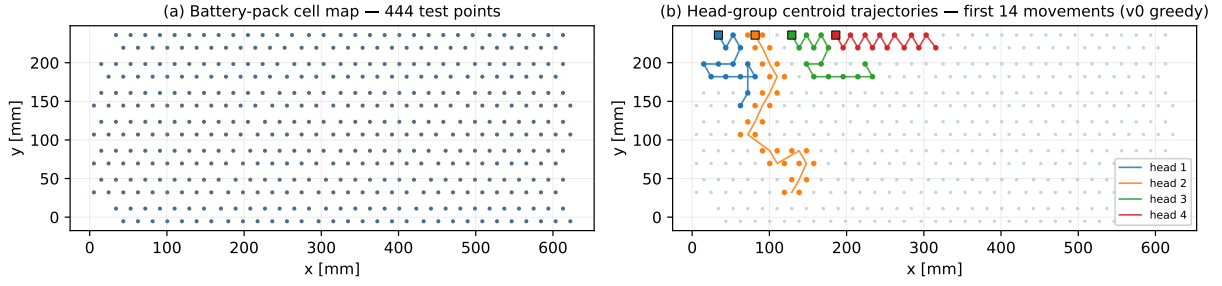


Figure 1: (a) The committed production dataset: 444 test points spanning 623×241 mm. (b) Centroid trajectories of the four heads over the first 14 movements of the committed baseline plan; squares mark starting placements, colours identify heads, and dots show the probed groups. Head strips advance left to right without crossing, as required by the mechanical constraint.

Table 2: Planner evolution (branch tips, 2019).

v	Date	Increment
0.0	Nov 21	Baseline greedy: nearest compatible group per head
1.0	Nov 25	Heads may idle within a movement
2.0	Dec 7	Whole-solution-space exploration
3.0	Nov 28	Exploration + idle heads
4.0	Dec 9	Merge of 2+3; multiple candidate choices per step
5.x	Dec 10	Refinement track (5.1.1.5.2)
7.0	Dec 15	Fully exhaustive search; backtracking; incumbent memorization
6.3.1	Dec 16	Final: multi-start over all feasible start configurations; phase-guided choices; idle-head partitioning; B&B; plan serialized to file

4.65 cells per movement, and ≈ 24.9 m of total centroid travel across the four heads. Figure 1 overlays the first 14 movements on the pack map: the four heads sweep parallel strips left to right, respecting the non-crossing order.

For the final v6 engine, the program’s own convergence note—recorded during development—reports that repeating the multi-start run 10 times yielded the optimal-cost plan in 98% of trials, with the remaining runs differing by a marginal amount of total travel; this is an empirical observation from the 2019 development environment, not a proven bound.

7 Validation and 2026 retrospective

Invariant checking. A separate Java harness (branch test) parses emitted plans and verifies the non-crossing invariant: within every movement, the x -coordinates of the probed groups must be strictly increasing across the head order.

Memory-safety audit (2026). Re-auditing the code with GCC and AddressSanitizer—six years and one toolchain later—surfaced three latent defects, all silently masked by the original MSVC stack/heap layout: (i) a stack buffer overflow in head parsing, where a 20-byte buffer was passed to an `fgets` bounded at 200 bytes and the actual header line is 21 bytes; (ii) a heap off-by-one in group enumeration, where the candidate vector was allocated with κ slots but also stores the -1 sentinel at index κ ; and (iii) a shifted row copy in deduplication, where a post-incremented index

left element 0 unwritten and dropped the sentinel, making set-equality read past the buffer and duplicate detection unreliable. With the three one-line fixes applied (committed as a single audit commit on master), v6.3.1 builds warning-clean and the full $\approx 10^4$ -start exploration runs to completion on Linux. Under the head-clearance constants committed at the end of the internship it terminates without a feasible plan—consistent with the repository’s T0 D0, which records that the physical clearance parameters were still being tuned with the machine designers when the internship ended; the committed baseline plans were produced under the earlier, wider tolerances.

8 Conclusions

Pylot-BT shows a complete path from an industrial requirement to a working combinatorial planner: a formalized constraint model, a dependency-free ADT library, hand-built enumeration and search machinery, and an explicit exactness/scalability trade-off resolved by measurement rather than assumption (v7’s exhaustive experiment directly motivated v6’s hybrid design). The 2026 audit closes the loop on engineering hygiene: the algorithms were sound, and the three latent memory defects—invisible on the original toolchain—were found and fixed with modern sanitizers, leaving the historical code buildable and runnable today.

Acknowledgements. Developed at Seica S.p.A. in coordination with the machine-design team. The dataset is an anonymized cell map used for development. “Pylot” is the project’s internal spelling of the Pilot machine family name.